

Rapport J17 - SQL : filtres de jointure et agrégations conditionnelles

Plan d'exécution SQL/Data - 16 semaines | Semaine 3

Synthèse

Parcours	Plan d'exécution SQL/Data - 16 semaines
Semaine	Semaine 3
Jour pédagogique	J17
Durée prévue	90 min
Durée réelle	99 min (1 h 39)
Écart prévu/réel	+9 min, soit +10 %
Base	PostgreSQL 17.5 - sql_training
État initial	4 sources / 15 événements
État final	4 sources / 15 événements

Objectifs

- Approfondir l'analyse relationnelle multi-table avec INNER JOIN et LEFT JOIN.
- Comprendre l'effet du placement d'un filtre dans ON ou WHERE.
- Construire des agrégations conditionnelles avec FILTER.
- Calculer un KPI de taux robuste avec NULLIF.
- Préserver correctement les sources sans événement.
- Construire un tableau de pilotage analytique avec CTE, CASE, AVG, SUM et COALESCE.
- Valider la robustesse par une transaction temporaire puis restaurer l'état permanent.
- Produire un script SQL reproductible exécutable avec ON_ERROR_STOP=1.

Notions clés

- **INNER JOIN + ON/WHERE:** Pour le filtre testé sur la table jointe, les deux placements ont produit le même ensemble final avec INNER JOIN : 7 événements energy.
- **LEFT JOIN + WHERE:** Un filtre sur la table droite dans WHERE peut éliminer les lignes NULL préservées par le LEFT JOIN.
- **LEFT JOIN + ON:** Le filtre dans ON limite les correspondances de la table droite tout en préservant toutes les lignes de la table gauche.
- **COUNT(*) vs COUNT(colonne):** COUNT(*) compte la ligne préservée d'un LEFT JOIN ; COUNT(ste.event_id) ignore NULL et représente donc correctement 0 événement.
- **FILTER:** Permet de compter conditionnellement les événements dont event_value >= 100 sans supprimer les autres lignes du groupe.
- **NULLIF:** Protège le dénominateur du KPI contre la division par zéro.
- **AVG et NULL:** AVG retourne naturellement NULL lorsqu'aucune observation n'existe ; il ne faut pas convertir arbitrairement une vraie moyenne de 0 en NULL.
- **SUM + COALESCE:** Pour total_value, l'absence d'événement est présentée comme 0.00.
- **CTE:** Le CTE source_activity sépare le calcul des agrégats de leur interprétation métier via CASE.
- **Transaction:** BEGIN/ROLLBACK permet de tester une source sans événement sans modifier durablement le dataset.

Déroulement et résultats

J17-01 - Baseline relationnelle

15 événements, 4 source_id distincts, event_id min=1, max=18 ; 4 sources, 4 actives, 4 codes distincts.

J17-02 - Filtre métier multi-table

Catégories energy/iot avec event_value >= 30 : 6 lignes.

J17-03 - INNER JOIN : filtre ON vs WHERE

Les deux variantes retournent 7 événements energy.

J17-04 - LEFT JOIN : filtre ON vs WHERE

WHERE : 6 lignes et 2 sources ; ON : 8 lignes, 6 événements qualifiants et 4 sources préservées.

J17-05 - Agrégation conditionnelle

manual_psql 1/1/125.50 ; microgrid_poc 7/5/190.99 ; sensor_gateway 4/0/28.00 ; tomra_training 3/0/3.00.

J17-06 - KPI high_value_rate_pct

100.00 %, 71.43 %, 0.00 %, 0.00 %. Division sécurisée avec NULLIF.

J17-07 - Exercice autonome

CTE + LEFT JOIN + FILTER + AVG + CASE. Corrections : retrait de NULLIF autour de AVG, CASE simplifié, ajout ORDER BY.

J17-08 - Challenge final

Tableau de pilotage complet. Correction de ROUND(SUM(...)) vers ROUND(SUM(...), 2) pour conserver 1336.90.

J17-09 - Test source sans événement

billing_api temporaire : total=0, high=0, moyenne=NULL, total=0.00, taux=NULL, statut=NO EVENTS. ROLLBACK ensuite.

J17-10 - Script reproductible

J17_join_filters_conditional_aggregation.sql exécuté avec -v ON_ERROR_STOP=1 ; fin : J17 COMPLETE.

Requête analytique finale

```
WITH source_activity AS (
    SELECT
        ss.source_code,
        ss.source_name,
        ss.source_category,
        COUNT(ste.event_id) AS total_events,
        COUNT(ste.event_id)
            FILTER (WHERE ste.event_value >= 100) AS high_value_events,
        ROUND(AVG(ste.event_value), 2) AS average_value,
        COALESCE(ROUND(SUM(ste.event_value), 2), 0.00) AS total_value,
        ROUND(
            100.0 * COUNT(ste.event_id)
                FILTER (WHERE ste.event_value >= 100)
            / NULLIF(COUNT(ste.event_id), 0),
            2
        ) AS high_value_rate_pct
    FROM analytics.source_systems AS ss
    LEFT JOIN staging.training_events AS ste
        ON ss.source_id = ste.source_id
    GROUP BY
        ss.source_code,
        ss.source_name,
        ss.source_category
)
SELECT
    source_code,
    source_name,
    source_category,
    total_events,
    high_value_events,
    average_value,
```

```

total_value,
high_value_rate_pct,
CASE
    WHEN total_events > 0 THEN 'ACTIVE'
    ELSE 'NO EVENTS'
END AS activity_status
FROM source_activity
ORDER BY high_value_rate_pct DESC NULLS LAST, source_code;

```

Erreurs et résolutions

- **NULLIF autour de AVG:** La première version utilisait NULLIF(ROUND(AVG(...),2),0), ce qui transformait une vraie moyenne de 0 en NULL. Correction : ROUND(AVG(...),2).
- **CASE total_events < 0:** COUNT ne peut pas être négatif. Correction : WHEN total_events > 0 THEN 'ACTIVE' ELSE 'NO EVENTS'.
- **Tri manquant J17-07:** Ajout de ORDER BY source_code.
- **Perte des décimales sur total_value:** ROUND(SUM(event_value)) arrondissait 1336.90 à 1337. Correction : ROUND(SUM(event_value), 2).

Captures à archiver

- J17-01_environment-and-relational-baseline.png
- J17-02_multitable-business-filter.png
- J17-03_inner-join-on-vs-where.png
- J17-04_left-join-on-vs-where.png
- J17-05_conditional-aggregation-by-source.png
- J17-06_conditional-rate-kpi.png
- J17-07_autonomous-relational-analysis.png
- J17-08_final-challenge.png
- J17-09_zero-event-source-transaction-test.png
- J17-09_rollback-and-permanent-state.png
- J17-10_reproducible-script-success-1.png
- J17-10_reproducible-script-success-2.png

Validation finale

- État permanent final : 4 sources, 4 sources actives, billing_api_remaining=0.
- staging.training_events : 15 événements et 4 source_id distincts.
- Le script complet atteint J17 COMPLETE sans erreur avec ON_ERROR_STOP=1.
- La transaction de robustesse est annulée par ROLLBACK.

KPI de progression

KPI	Résultat
Étapes pédagogiques	J17-01 à J17-10 validées
Dataset final	4 sources / 15 événements
Filtre métier	6 lignes qualifiantes
INNER JOIN ON vs WHERE	7 = 7
LEFT JOIN WHERE	6 lignes / 2 sources
LEFT JOIN ON	8 lignes / 6 événements / 4 sources
Taux high-value	manual_psql 100.00 %, microgrid_poc 71.43 %, autres 0.00 %
Robustesse zéro événement	Validée avec billing_api temporaire
Reproductibilité	Script complet : succès

KPI	Résultat
Temps	99 min réalisés / 90 min prévus / +9 min

Git / commit

- Le script est situé sous documentation/semaine-03/jour-17/code/J17_join_filters_conditional_aggregation.sql.
- Les rapports MD/DOCX/PDF et les captures doivent être placés sous documentation/semaine-03/jour-17/.
- Étape J17-12 encore à effectuer : git status, git add, validation du staging, commit, push, puis contrôle main = origin/main.
- Message de commit proposé : docs: complete J17 SQL join filter semantics and conditional aggregation

Potentiel de réutilisation

- Tutoriel : différence ON vs WHERE avec INNER JOIN et LEFT JOIN.
- Vidéo éducative : pourquoi COUNT(*) peut être trompeur après un LEFT JOIN.
- Atelier : construire un KPI conditionnel robuste avec FILTER et NULLIF.
- Portfolio : tableau de pilotage par source avec gestion explicite des sources sans événements.
- Formation : exercice transactionnel reproductible montrant BEGIN/ROLLBACK et la robustesse des agrégats.

Conclusion

J17 consolide la compréhension des jointures relationnelles, du placement des filtres et des agrégations conditionnelles. Le point majeur est la capacité à préserver les entités sans faits associés tout en calculant des KPI sémantiquement corrects. Le test transactionnel et le script reproductible confirment que la logique est robuste et que l'état permanent de la base est préservé.

Prochaine étape

Après la clôture Git de J17, poursuivre séquentiellement avec J18. Les journées non clôturées ne doivent pas être sautées pour rejoindre artificiellement le jour calendaire.